

Labwork 4: Threads

Vu Ha Vy – 2540112

October 1, 2026

1 2D CUDA Implementation

In Labwork 3, the image was flattened into a 1D array, but in Labwork 4, threads are used in a 2D grid that maps directly to image coordinates $(x, y) = (\text{width}, \text{height})$. The remaining pipeline is the same as in Labwork 3, where the image is transferred to device memory, processed by the kernel, and then copied back to host memory.

```
@cuda.jit
def grayscale(src, dst):
    tidy = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    tidy = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    g = np.uint8((src[tidy, tidy, 0] + src[tidy, tidy, 1] + src[tidy, tidy, 2]) / 3)
    dst[tidy, tidy, 0] = dst[tidy, tidy, 1] = dst[tidy, tidy, 2] = g
```

2 Qualitative Results



Figure 1: Original input image.

Fixed Grid Size

This experiment uses a fixed grid size of $\text{gridSize} = (8, 8)$ with $\text{blockSize} = (32, 32)$, as in the lecture slide.

The total number of threads is $8 \times 32 = 256$ in width and $8 \times 32 = 256$ in height. If the image dimensions exceed 256×256 , we can see that only a portion of the image is processed.

Dynamic Grid Size

To ensure full coverage of the image, the grid dimensions are computed based on the image's height and width:

```
blockSize = (32, 32)
gridSize = (math.ceil(width / blockSize[0]), math.ceil(height / blockSize[1]))
grayscale[gridSize, blockSize](devSrc, devDst)
```

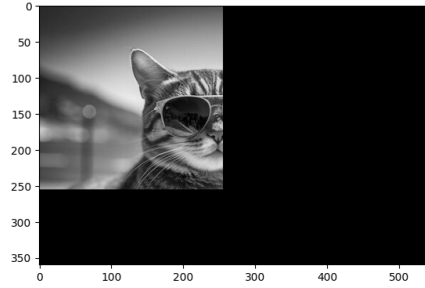


Figure 2: Fixed grid size output.

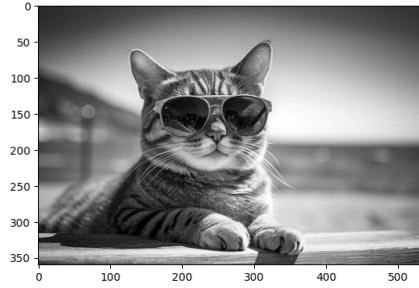


Figure 3: Dynamic grid size output.

3 Quantitative Results

Experiments were tested across different 2D block configurations and compared against the 1D kernel execution times from Labwork 3.

Threads/ 1D Block	2D Block	Grid Size	1D Time (s)	2D Time (s)
64	(8, 8)	(68, 45)	0.001091	0.001621
128	(8, 16)	(68, 23)	0.003341	0.001333
256	(16, 16)	(34, 23)	0.001880	0.000939
512	(16, 32)	(34, 12)	0.001700	0.000998

Table 1: Running Time Comparison Between 1D and 2D CUDA Kernels

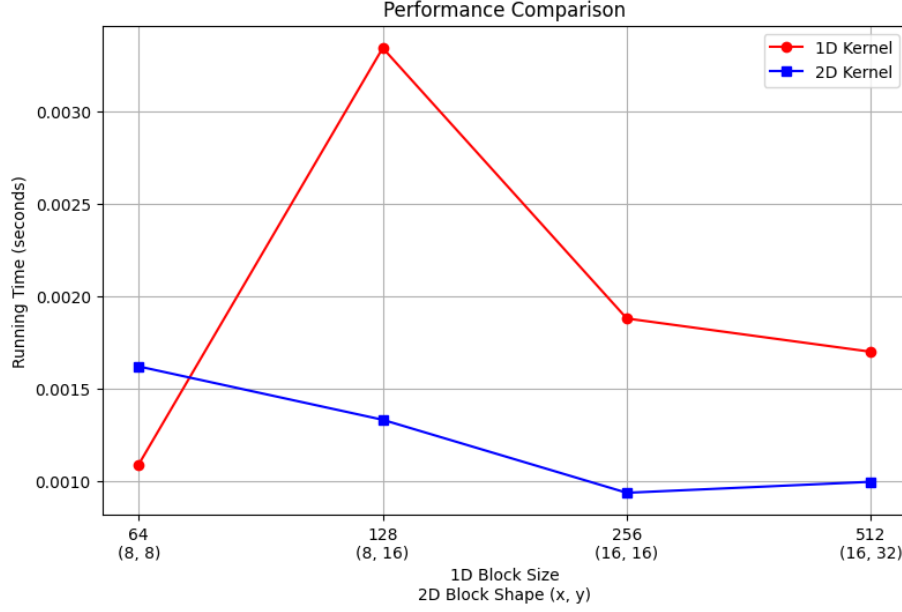


Figure 4: Running Time Comparison Between 1D and 2D CUDA Kernels

In the 2D implementation, when the block size increase from (8, 8) with 64 threads to (16, 16) with 256 threads, processing time drops from 0.001621 s down to a minimum of 0.000939 s. This improvement occurs because larger blocks use the hardware resources of the GPU more effectively. However, increasing the size further to (16, 32) causes execution time to rise slightly to 0.000998 s, indicating that (16, 16) provides the optimal balance between thread count and resource limits per block.

When comparing the two implementations, the 2D kernel generally outperforms the 1D version across most block configurations. Except for the smallest size of 64 threads where the 1D kernel is faster (0.001091 s vs 0.001621 s). The 2D kernel achieves speedups 1.70x to 2.51x for larger configurations. Mapping threads directly in two dimensions allows the GPU to access image pixel data much more efficiently in memory compared to the flattened 1D approach.

4 Exercises

Exercise 1

- 8×8 (64 threads): The SM has 8 blocks ($8 \times 64 = 512$ threads total), so it only runs at 50% occupancy.
- 16×16 (256 threads): We can use 4 blocks, so that $4 \times 256 = 1024$ threads total, giving full 100% occupancy.
- 32×32 (1024 threads): Limit of the GPU is 512 threads per block.

Answer: 16×16

Exercise 2

- 128 threads/block: Using 4 blocks, we have $4 \times 128 = 512$ threads total.
- 256 threads/block: Using 4 blocks, we have $4 \times 256 = 1024$ threads total.
- 512 threads/block: Using 3 blocks, we have $3 \times 512 = 1536$ threads total, reaching maximum thread capacity.
- 1024 threads/block: Using 1 block, we have $1 \times 1024 = 1024$ threads total.

Answer: 512 threads/block

Exercise 3

To process 2000 elements with a block size of 512, we need $\lceil 2000/512 \rceil = 4$ blocks. Running 4 blocks gives $4 \times 512 = 2048$ total threads. So 2000 will do the actual work, and the remaining 48 threads will wait.

Answer: 2048 threads